

5, and the compendium version is 4.3. Each part of the specification is owned by a voluntary expert group, and there is a board of directors that oversees expert groups. Apart from writing down specifications, the OSGi Alliance also creates reference implementations for the specification and test suites. It also certifies any new implementation for compliance with the specification.

OSGi bundle

The basic unit of deployment in OSGi is called a *bundle*. An OSGi application is composed of multiple bundles. A bundle is a self-contained unit implementing a specific functionality; it works in conjunction with other bundles, and it shares and consumes functionality with or from other bundles. Bundles serve two important purposes—modularity and reusability. A bundle is a JAR file with a manifest containing metadata about the bundle. Bundles declare their dependencies in the manifest file, and the container takes care of wiring them at run time. Even the dependencies could be versioned, and multiple versions can co-exist. In OSGi, every class is bundle-private, and through its manifest, a bundle can specify which package it shares using the *export* directive; other modules can use it by an *import* directive. The container takes care of delegating the class loading requests between bundles. Each bundle has a lifecycle, which starts with the installed state, and ends once it passes the uninstalled state.

Another important aspect of OSGi is the class loading. Each module has a class loader, and unlike hierarchical class loaders in traditional Java applications, OSGi class loaders work as a network. Since OSGi is a dynamic module system, modules can be dynamically installed or uninstalled, and the container takes care of notifying the dependent bundles. The OSGi container passes the bundle's context to all the bundles, which is useful to query and manipulate the state of the container and the bundles running.

OSGi also has a concept of declarative services, similar to Web services. Any bundle can expose any number of services, and other bundles may consume them. OSGi services are registered in a service tracker, and bundles can look up the tracker and consume services accordingly. The difference between OSGi services and Web services is the absence of the network in the former. Since OSGi services and their consumers run in the same container, there is no network layer involved.

The OSGi framework

The OSGi framework has been developed as per the specifications published by the OSGi Alliance. It runs over JVM and provides a secure environment for the bundle to get deployed, run and manage the inter-bundle dependencies. It is made of the execution environment, bundles, service registry and bundle life-cycle management components. The OSGi framework controls the life-cycle of a bundle.

OSGi implementations

Since OSGi is an open standard, there are many implementations available. Equinox has been the reference implementation for OSGi for quite some time. The current specification implemented by OSGi is the Core R4 version. It also enjoys support from the Eclipse Foundation, as the Eclipse plug-in system is built on top of Equinox. There are several popular open source implementations like Felix, Knopplerfish and Concierge. The last one is a compact and optimised implementation suited for mobile platforms, but it conforms to R3 specifications only. Felix is a promising implementation, as it is compact and lightweight when compared to Equinox.

Modularity support in Java

Initially, the Java language was not designed for modularity—but attempts were made later to include modularity. Some of the notable Java Specification Requests (JSR) for bringing modularity are JSR 277, 291 and 294. JSR 277 defines a static module system in Java, as opposed to OSGi's dynamic module system. JSR 291 defined a dynamic module system, but it was just a reference to the OSGi R4 specification, and did not pass through the normal JCP process, where an expert group debates and finalises the specifications. JSR 294 is an effort to add language and JVM support for module systems. Project Jigsaw is the reference implementation for JSR 294 from OpenJDK. An early-access J2SE 8 release with Project Jigsaw is available from the OpenJDK website, and full-blown Jigsaw may feature in future releases.

Rise of Equinox

When Eclipse's own plug-in system was facing immense challenges in keeping up to what was required of it, a total revamp was called for. Project Equinox was started by the Eclipse development team to look at possible alternatives for Eclipse's runtime. Avalon, JMX, and OSGi were studied carefully, and OSGi won the race—mainly because of its dynamic nature. The Equinox development team collaborated with the OSGi core specification expert group by providing inputs for the specification. The team also started implementing the specification, and that is how Equinox was born. Some of the key specification improvements that came through the Equinox team were bundle fragments, lazy loading of bundles, and *require-bundle* directives. They were not part of the R3 specification, but made it into the R4 specification. Equinox is currently the most-used implementation of OSGi, and the current version of Equinox corresponds to the Core R4 OSGi specification. Equinox does not implement the complete OSGi specification; it stops with just core specifications. Others, like the compendium, mobile, enterprise and vehicular specifications are not implemented.

How Eclipse is built around Equinox

Eclipse can be considered as a group of bundles running on the Equinox OSGi runtime. Prior to version 3.0, Eclipse was